Додаткова технічна AI-сесія: повідомлення R

Технічні витяги про виконану розробку html2json та перевірки.
Залишено результати створення коду, незалежних аудитів, тестування й виправлень.
Це вибірка з відредагованого джерела, а не повний оригінальний експорт чату.
Повідомлення перекладено українською. Пропущено організаційні бесіди та чернетки промптів.

R231 · Кандидат

Інтерфейс джерела: «Додано зображення».

https://github.com/jito-dev/jito-intern-test-task

Саме зображення в share-представленні не відображено. У наступній відповіді AI зазначає, що на ньому була персональна форма здачі; такого персонального URL у цьому документі немає.

R255 · Кандидат

Інтерфейс джерела: «Додано зображення» × 12.

Самих зображень у share-представленні не відображено. Нижче — весь видимий технічний текст повідомлення, без службового «Развернуть»; кодові приклади й JSON не змінено.

```
function html2json(htmlText) {
// HTML string → JSON-compatible object
} <p class="intro">
Hello <strong>world</strong>!
</p> {
"type": "document",
"children": [
{
"type": "element",
"tagName": "p",
"attributes": {
"class": "intro"
},
"children": [
{
"type": "text",
"value": "Hello "
},
{
"type": "element",
"tagName": "strong",
"attributes": {},
"children": [
{
"type": "text",
"value": "world"
}
]
},
{
"type": "text",
"value": "!"
}
]
}
```

```
]
} <ul>
<li>First
<li>Second
</ul>
```

R258 · AI

Аміго, першу версію вже можна назвати робочою, але надсилати її jito.dev поки рано. Я не обмежився скриншотами: розпакував твій ZIP, вивчив html2json.js, тести, документацію й самостійно запустив перевірки.

Результати незалежного аудиту

- Автоматичні тести — 27/27 PASS.
- Перевірка синтаксису.
- Тестові HTML-файли — 14.
- Вихідний інтерфейс — збережено.
- Публічне посилання на AI-діалог — ще не додане.

Підтверджую: усі 27 тестів пройшли в Node.js 22.16.0, а `npm run check` завершився без помилок. Однак тести перевіряють лише передбачені сценарії.

Я вже виявив справжню проблему, якої ці 27 тестів не виявили.

Проблема №1 — суттєве сповільнення на незвичайному HTML

Я самостійно перевірив, як парсер працює з повторюваними незавершеними атрибутами:

```
const html = '<a title="'.repeat(9000);
html2json(html);
```

Вимірювання в моєму середовищі, не результати роботодавця.

В оригінальному інтерактивному графіку текстово показані поділки 0 мс; 800 мс; 1,6 тис. мс; 2,4 тис. мс; 3,2 тис. мс та розміри вводу 30, 60 і 90 тис. символів. Час виміряно локально на Node.js 22.16.0. Точні числа наведено й у створеному нижче промпті; їх не видаємо за нове вимірювання.

Зі збільшенням розміру вводу втричі час зріс приблизно в дев'ять разів. Це свідчить про близьку до квадратичної поведінку на такому класі вхідних даних.

Причина: коли відкривальний тег не завершений, `readStartTag()` може багаторазово переглядати решту тексту. Зовнішній цикл потім пробує почати розбір із наступного символу `<`, і робота повторюється.

Для завдання, де окремо оцінюють стійкість до неочікуваного вводу, це суттєвий ризик. Потрібно виправити причину повторних проходів, а не просто збільшити обмеження на розмір документа.

Ще два моменти, які потрібно виправити або перевірити

2. Покриття тестами поки недостатньо суворе. Тести для 14 HTML-файлів перевіряють, що результат серіалізується й має кореневий тип document. Але для більшості цих файлів немає окремої перевірки очікуваної структури дерева. Отже, деякі логічні помилки можуть залишатися непоміченими.

3. Є задокументовані обмеження HTML-сумісності. Наприклад, `é` і `♥` поки не декодуються, а SVG-атрибут `viewBox` перетворюється на `viewbox`. Це не обов'язково порушення завдання, але потрібно свідомо визначити, які випадки варто підтримати, і не обіцяти більше, ніж уміє реалізація.

Окремо я перевірив функції інтерфейсу в Chromium із підстановкою HTML і JavaScript безпосередньо у сторінку: обидві кнопки прикладів та конвертація спрацювали, HTML із поля не виконався як JavaScript. Повне

завантаження сайта через локальний HTTP-сервер у цьому середовищі перевірити не вдалося через блокування доступу браузера, тому таку перевірку ще варто виконати окремо.

[Подальший згенерований запит для іншої сесії тут не наведено: це не підтвердження виконаної зміни коду.]

R260 · AI

Аміго, згоден. Ці формулювання краще прибрати. Вони виглядають як витяг з аудиторського звіту, а не як звичайний запит розробника до AI.

Точні мілісекунди теж не потрібні. Нам важливо, щоб ChatGPT самостійно відтворив проблему, встановив причину й перевірив виправлення.

Водночас немає потреби в кожному технічному запиті пояснювати, звідки виникла ідея тесту. Але під час фінальної здачі потрібно чесно виконати вимогу jito.dev про розкриття історії використання AI.

Ось як я б написав наступний промпт

Український оригінал створеного запиту; нормалізовано лише розмітку.

Дякую, перша версія виглядає добре. Але перед фінальною підготовкою хочу ще ретельніше перевірити стійкість парсера.

Давай почнемо з продуктивності. Цікавить, як він поводиться з великими обсягами некоректної розмітки, особливо з незавершеними тегами й атрибутами.

Наприклад, перевір такий випадок:

```
const html = '<a title="'.repeat(9000);
html2json(html);
```

Перевір декілька розмірів такого вводу, виміряй час виконання та подивися, чи немає зайвих повторних проходів по рядку. Якщо знайдеш проблему, виправ її причину й додай регресійний тест.

Також хочу посилити перевірки правильності JSON. Недостатньо лише переконатися, що парсер не падає — для основних HTML-прикладів потрібно перевіряти очікувану структуру, вкладеність, порядок вузлів, атрибути й текст.

Окремо зверни увагу на HTML-сутності та SVG-атрибути, зокрема é, ♥ і viewBox. Якщо щось залишається поза межами підтримки, це потрібно чітко пояснити в документації.

Після виправлень повторно запусти всі тести, перевір, що попередня функціональність не зламалася, і підготуй оновлений архів. Поки нічого не публікуємо.

Чому цей варіант кращий

- Звучить природно, без штучної експертності.
- Не містить непотрібних посилань на інший чат.
- Не видає вимірювання за твої особисті результати.
- Зберігає всі важливі технічні перевірки.
- Дозволяє отримати справжні докази виправлень.

І головне: ми не повинні перетворювати історію на виставу для перевіряльника. Нормально, коли кандидат використовує AI, ставить запитання, перевіряє результат і поліпшує код — компанія саме цього й очікує.

Я рекомендую надіслати саме цей варіант. Він коротший, природніший і зберігає технічну цінність попереднього запиту.

R262 · AI

Незалежна перевірка версії v2

- Автоматичні тести — 49/49 PASS.
- Синтаксис JavaScript.
- HTML-приклади — 16 файлів.
- Виправлення сповільнення — підтверджене.

Я самостійно розпакував архів `jito-html2json-stability-audit-v2.zip` і запустив `npm test`, `npm run check` та `npm run benchmark` у Node.js 22.16.0.

Попередню проблему справді виправлено: тест із 9 000 незавершених атрибутів тепер обробляється за частки мілісекунди після прогрівання, замість кількох секунд. Із подальшим збільшенням розміру вводу час зростає набагато передбачуваніше.

Також підтверджено, що кількість тестів зросла з 27 до 49, з'явилися точні перевірки JSON-структури для HTML-прикладів, а не лише перевірки відсутності винятків.

Це суттєве поліпшення. Але до здачі потрібно ще оцінити, чи не з'явилися помилки на інших складних конструкціях HTML.

Що виявила додаткова перевірка

Виправлення продуктивності працює, але я знайшов кілька нових випадків, де парсер будує дерево інакше, ніж браузер.

1. Вкладені заголовки

Вхід:

```
<h1>First<h2>Second</h2>
```

Поточна версія розміщує h2 усередині h1. Браузерний HTML-парсер у цій ситуації закриває попередній заголовок, тому елементи стають сусідніми. [Посилання оригіналу: `html.spec.whatwg.org`.]

2. Абзац перед елементом списку

Вхід:

```
<p>Before<li>First</li>
```

Поточна версія залишає li усередині p, тоді як браузер закриває абзац перед обробкою li. [Посилання оригіналу: `html.spec.whatwg.org`.]

3. Спеціальні текстові елементи

Наприклад:

```
<plaintext>A<b>B</b>&amp;C
```

У браузері після `<plaintext>` вміст трактується як звичайний текст, а поточна версія створює вкладений елемент b. Схожі відмінності існують для xmp та iframe.

Це не означає, що потрібно терміново писати повний HTML5-парсер: README вже чесно попереджає про обмеження. Але такі випадки варто оцінити перед фінальним надсиланням.

Наступний запит у публічний чат

Український оригінал створеного запиту; нормалізовано лише розмітку.

Дякую, добре, що проблему з продуктивністю вдалося виправити.

Хочу ще трохи перевірити поведінку парсера на незвичних, але реальних HTML-конструкціях, перш ніж готувати фінальну версію.

Зокрема, подивись на такі приклади:

```
<h1>First<h2>Second</h2>
```

```
<p>Before<li>First</li>
```

```
<button>A<button>B</button>
```

```
<plaintext>A<b>B</b>&amp;C
```

Також перевір обробку вмісту xmp та iframe.

Порівняй результат нашого парсера з правилами HTML Standard. Якщо є розбіжності, оціни, які з них варто виправити без суттєвого ускладнення архітектури, а які краще залишити документованими обмеженнями.

Не хочу без потреби розширювати проєкт до повного HTML5-парсера. Пріоритет — коректність поширених випадків, зрозумілий код і відсутність регресій.

Для внесених виправлень додай точні тести структури JSON та повторно запусти весь набір перевірок.

Окремо спробуй перевірити початковий index.html у справжньому браузері. Якщо середовище не дозволяє, не позначай цю перевірку як успішну.

Після цього надай оновлений архів і коротко зазнач, які обмеження залишилися. Поки нічого не публікуємо.

Що ще не готове до здачі

В архіві `ai_help/chatgpt_chat.txt` поки міститься заглушка замість посилання на робочу історію ChatGPT. Крім того, повноцінну повторну перевірку сторінки в Chromium не було підтверджено: спроба запуску завершилася тайм-аутом.

Це два пункти для фінального етапу, а не привід зупиняти виправлення просто зараз.

Моя оцінка: версія v2 стала значно кращою за першу. Тепер ми перевіряємо вже тонкощі обробки HTML, а не виправляємо базову працездатність. Після наступної ітерації варто провести заключний аудит і, якщо серйозних дефектів не залишиться, переходити до підготовки репозиторію.

R264 · AI

Аміго, бачу третю версію: заявлені 60 пройдених тестів, виправлення за HTML Standard і новий архів. Перевірю сам ZIP, повторю тести й окремо подивлюся на розбіжності з браузерним розбором HTML. Після цього скажу, чи готовий проєкт до публікації, чи потрібен ще один точковий прохід.

R268 · AI

Аміго, продовжую з останньої зафіксованої точки — незалежний аудит версії v3 технічного завдання jito.dev. Нічого заново не починаємо й роботодавцю поки не надсилаємо.

Я розпакував `jito-html2json-standards-audit-v3.zip` і повторно запустив перевірки на Node.js 22.16.0.

- Автоматичні тести — 60/60 PASS.
- Перевірка синтаксису.
- Тестові HTML-файли — 22.
- Проблема зі сповільненням — виправлена.
- Посилання на AI-діалог — поки заглушка.

Особливо важливо: тест із 36 000 повторень незавершеного атрибута (360 000 символів) тепер обробляється приблизно за 3,1 мс у моєму середовищі. Попереднє багаторазове сканування рядка справді усунено.

На твоєму останньому скриншоті інтерфейс правильно показує два сусідні заголовки h1 і h2, а не помилкову вкладеність. Це підтверджує виправлення одного з раніше знайдених дефектів.

Однак 60 успішних тестів ще не гарантують проходження прихованих перевірок компанії. Тому перевіряю залишкові ризики в самій реалізації, а не лише її тестовий звіт.

Результат додаткової перевірки Chromium

Я перевірив 38 різних HTML-фрагментів, порівнявши дерево нашого конвертера з результатом браузерного HTML-парсера.

- Збіглися за структурою — 32 із 38.
- Виявлені відмінності — 6.

Відмінності стосуються вставлення tbody у таблиці, відновлення неправильної вкладеності тегів b та i, видалення початкового переводу рядка в pre, перенесення тексту з таблиці та вкладених form.

Це не шість нових критичних дефектів. Більшість таких особливостей уже належать до обмежень, прямо описаних у README. Наша функція не обіцяє точної копії браузерного HTML5-парсера.

Я також перевірив у Chromium, що кнопка Convert to JSON працює, а обидві кнопки Input Example заповнюють вихідні приклади. Перевірка виконувалася з підстановкою оригінальних HTML і JavaScript у браузер. Відкрити сторінку безпосередньо через `file://` або локальний HTTP-сервер у моєму середовищі не вдалося через обмеження доступу.

Що я рекомендую зараз

Зупинити розширення HTML5-сумісності й перейти до фінальної перевірки. Додаткові складні правила можуть збільшити ризик регресій та ускладнити захист коду.

Ось наступний запит в український робочий чат.

Український оригінал створеного запиту; нормалізовано лише розмітку.

Схоже, ми вже виправили основні знайдені проблеми. Хотілося б тепер не ускладнювати парсер без потреби, а перевірити, чи готовий проєкт до подачі.

Проведи, будь ласка, фінальний аудит поточної версії. Звір усі файли з вимогами оригінального README jito.dev, повторно перевір тести, роботу інтерфейсу та відповідність документації фактичній поведінці.

Особливо уважно перевір, щоб ми ніде не заявляли повну HTML5-сумісність. Мені важливо, щоб усі відомі

обмеження були описані чесно, зокрема особливості таблиць, некоректної вкладеності та інших складних конструкцій.

Також перевір, чи немає зайвих файлів, тимчасових результатів, неправдивих тверджень про проведені перевірки або недоречних матеріалів у фінальному репозиторії.

Окремо підготуй точний список того, що залишається зробити вручну перед відправленням: перевірка в моєму браузері, публікація GitHub-репозиторію, додавання посилання на AI-діалог і перевірка доступності всіх посилань.

Якщо знайдеш конкретний дефект, виправ його і повтори регресійні тести. Якщо критичних проблем немає, не переписуй код заради додаткових покращень.

Після аудиту надай фінальний архів та чітко розділи підтверджені перевірки, відомі обмеження й незавершені дії. Поки що нічого не публікуй і не відправляй роботодавцю.

Поточний статус

- Основна реалізація готова.
- 60/60 автоматичних тестів пройдено незалежно.
- Раніше знайдене сповільнення усунено.
- Вихідний `index.html` збігається з файлом роботодавця за Git SHA.
- Фінальна перевірка на твоєму ПК й оформлення GitHub ще попереду.
- Публічне посилання на повну історію AI поки не додане.

Мій висновок: версія v3 достатньо якісна, щоб переходити до фінального аудиту й підготовки здачі. Зараз головний ризик — не відсутність ще десятка HTML5-функцій, а формальна помилка під час публікації або нездатність пояснити ключові частини алгоритму на code review.

Тому після фінального аудиту пропоную більше не нарощувати функціональність без виявленого дефекту, а зайнятися перевіркою репозиторію й твоєю підготовкою до технічного захисту.

R270 · AI

Аміго, фінальний архів справді проходить базові перевірки. Я розпакував `jito-html2json-audited-pre-submission.zip` і самостійно виконав команди на Node.js 22.16.0.

Перевірка	Результат
npm test	60/60 PASS
npm run check	PASS
npm run benchmark	Виконано
HTML-приклади	22 файли
Обов'язкові вихідні файли	На місці
ai_help/chatgpt_chat.txt	Поки заглушка

Перевірка продуктивності також виглядає добре: проблемний ввід довжиною 90 000 символів у моєму запуску обробляється приблизно за 0,7 мс після прогрівання.

Однак перед публікацією я вирішив додатково перевірити узгодженість токенізатора й побудови дерева. Саме в таких місцях можуть ховатися помилки, яких власні тести не охоплюють.

Знайдено ще одну справжню помилку

Перевірка виявила випадок, якого немає серед 60 тестів: самозакривальні спеціальні елементи всередині SVG.

Наприклад:

```
<svg>
```

```
<script/>
<circle cx="1"/>
</svg>
```

Я порівняв результат поточного `html2json()` із Chromium.

Chromium: створює два сусідні елементи — script і circle.

Наш парсер: створює script, але потім помилково сприймає решту розмітки як текст і втрачає елемент circle.

Аналогічна проблема відтворюється для `<style/>` і `<title/>` усередині SVG, а також для `<style/>` усередині MathML.

Причина: токенізатор вмикає спеціальний режим читання тексту після script, style або title, не враховуючи, що елемент у SVG вже самозакрився.

Це не привід переписувати весь проєкт. Помилка локальна, але виправити її до публікації варто: вона призводить до втрати справжньої структури документа.

Наступний крок: одне точкове виправлення

Зараз я не рекомендую надсилати фінальний архів на GitHub. Спочатку усунемо знайдену помилку й перевіримо, чи не зачепило виправлення звичайні HTML-документи.

Ось наступний запит для українського робочого чату — без громіздкого аудиторського звіту.

Український оригінал створеного запиту; нормалізовано лише розмітку.

Перевірив фінальний варіант і хочу уточнити ще один момент перед публікацією.

Зверни увагу на спеціальні елементи всередині SVG та MathML. Наприклад:

```
<svg><script/><circle cx="1"/></svg>
```

```
<svg><style/><circle/></svg>
```

```
<svg><title/><circle/></svg>
```

Схоже, після самозакривального script, style або title парсер іноді сприймає решту документа як звичайний текст і втрачає наступні елементи.

Також перевір випадки, коли такі елементи не є самозакривальними:

```
<svg><script><circle/></script><circle/></svg>
```

Порівняй поведінку з Chromium і визнач причину. Важливо врахувати різницю між звичайними HTML-елементами та елементами в SVG/MathML.

Якщо проблема підтвердиться, виправ її без зайвого ускладнення архітектури. Обов'язково перевір, щоб звичайний HTML script, style, textarea і title продовжували працювати правильно.

Додай регресійні тести для виправлених випадків, запусти весь набір перевірок і переконайся, що початковий інтерфейс не зламався.

Після цього підготуй оновлений ZIP. Якщо ж певна поведінка потребує повноцінного HTML5-парсера, поясни це окремо — не потрібно переписувати весь проєкт заради рідкісного випадку.

Поки нічого не публікуємо.

Після цього

Пропоную більше не займатися нескінченним розширенням підтримки HTML5. Якщо це виправлення проходить регресійні тести й не виявляються нові суттєві дефекти, переходимо до завершальних дій:

1. Ручна перевірка на твоєму ПК — відкрити index.html, перевірити обидві кнопки прикладів і конвертацію.
2. Розмістити файли в окремому публічному GitHub-репозиторії.
3. Додати справжнє посилання на AI-переписку замість заглушки, дотримуючись вимоги компанії щодо повноти історії.
4. Перевірити посилання в інкогніто й повторити тести.
5. Лише після цього заповнити форму jito.dev.

Підсумок: версія вже достатньо зріла — 60 тестів проходять, продуктивність поліпшена, документація стала точнішою. Ми знайшли ще один конкретний дефект і можемо виправити його адресно. Наша мета зараз — не збільшити кількість тестів заради цифри, а виключити відомі помилки перед здачею.

R272 · AI

Аміго, версія v4 пройшла незалежну перевірку за основними критеріями. Виправлення помилки з SVG/MathML підтверджено не лише звітом із робочого чату, а й моїми запусками.

Результати перевірки v4

- Автоматичні тести — 70/70 PASS.
- Синтаксис JavaScript.
- HTML-приклади в архіві — 31.
- Тести продуктивності — пройдені.
- Виправлення SVG/MathML — підтверджене.
- Публікація на GitHub — ще не виконана.

Я незалежно запустив `npm test`, `npm run check` та `npm run benchmark` у Node.js 22.16.0. Усі 70 тестів пройшли.

Проблемний приклад:

```
<svg><script/><circle cx="1"/></svg>
```

тепер створює два сусідні SVG-елементи — script і circle, як і потрібно для цього випадку.

Також я перевірив роботу трьох кнопок у Chromium, програмно завантаживши вихідні HTML і JavaScript: обидві демонстрації та конвертація працюють, помилок виконання немає.

Обмеження перевірки: відкрити `index.html` безпосередньо через `file://` або локальний HTTP-сервер у моєму середовищі не вдалося — доступ блокується. Тому нам усе ще потрібен звичайний запуск на твоєму комп'ютері.

Що залишилося перед здачею

На цьому етапі я не рекомендую розширювати функціональність парсера без нового підтвердженого дефекту. Додаткові зміни можуть внести регресії.

Залишилися практичні дії:

1. Розпакувати саме `jito-html2json-foreign-content-fixed-v4.zip` на своєму комп'ютері.
2. Відкрити `index.html` у Chrome й перевірити обидві кнопки Input Example, потім Convert to JSON. Окремо вставити SVG-приклад і переконатися, що в JSON є script та circle.

3. Додати справжнє посилання на робочий AI-діалог до `ai_help/chatgpt_chat.txt`. Водночас дотриматися вимоги jito.dev щодо повної історії використання AI й не розкривати особистих відомостей.
4. Розмістити перевірені файли в окремому публічному GitHub-репозиторії.
5. Перевірити репозиторій та посилання на AI-діалог у режимі інкогніто, повторити тести й лише потім здавати результат.

[Подальший згенерований запит для іншої сесії тут не наведено: це не підтвердження виконаної зміни коду.]

R274 · AI

Аміго, зараз перевіримо саме останню версію v4 на твоєму комп'ютері з Windows і Google Chrome. Я вже звірив структуру архіву та результати роботи парсера на контрольних прикладах, тому покажу, що саме натискати й які результати мають з'явитися.

Наша мета — перевірити не лише те, що сторінка відкривається, а й що кнопки працюють, JSON формується правильно, складні випадки не ламають застосунок, а автоматичні тести проходять.

Крок 1. Завантаж і розпакуй правильний архів

Потрібен саме:

`jito-html2json-foreign-content-fixed-v4.zip`

Це версія з останнім виправленням SVG/MathML і 70 автоматичними тестами.

1. У публічному робочому чаті знайди повідомлення з назвою Jito HTML2JSON — Foreign Content Fix v4.
2. Натисни «Завантажити оновлений архів v4».
3. Відкрий папку Завантаження у Windows.
4. Натисни правою кнопкою миші на ZIP → Видобути все / Extract All.
5. Відкрий розпаковану папку.

Усередині мають бути приблизно такі файли:

```
README.md
index.html
html2json.js
package.json

ai_help/
benchmarks/
html_samples/
tests/
```

Я перевірив: в архіві v4 справді є ці файли та 31 HTML-приклад.

Важливо: не відкривай `index.html` із вікна перегляду ZIP-архіву. Спочатку видобудь усі файли, інакше Chrome може не знайти `html2json.js`.

Крок 2. Відкрий застосунок у Chrome

У розпакованій папці знайди `index.html`.

Натисни на нього правою кнопкою → Відкрити за допомогою → Google Chrome.

Має відкритися сторінка із заголовком:

В оригінальному інтерфейсі тут показано ілюстрацію з атрибуцією «sourcecodester». У текстовому UI-експорті зображення не відтворене; його не замінено вигаданим скриншотом.

Орієнтуйся передусім на назву сторінки та три кнопки, а не на оформлення зображення.

Ти маєш побачити два великі текстові поля й три кнопки:

- Convert to JSON
- Input Example 1
- Input Example 2

Якщо Chrome показує звичайну просту білу сторінку без сучасного дизайну — це нормально. Саме такий інтерфейс надала компанія.

Крок 3. Перевіряємо три кнопки

1. Input Example 1

Натисни Input Example 1.

Ліворуч має з'явитися великий HTML-документ із DOCTYPE, html, head, body та іншими тегами.

Потім натисни Convert to JSON.

Праворуч має з'явитися JSON із кореневим `"type": "document"` та вкладеними елементами.

2. Input Example 2

Натисни Input Example 2, потім Convert to JSON.

Праворуч має з'явитися JSON, що містить елементи div, p, button та textarea.

3. Convert to JSON

Видали вміст лівого поля, встав:

```
<p>Hello world!</p>
```

Натисни Convert to JSON. Праворуч мають бути `"tagName": "p"` та `"value": "Hello world!"`.

Якщо всі три кнопки працюють, перший етап ручної перевірки пройдено.

Крок 4. Перевіряємо складні випадки

Тепер по черзі вставляй у ліве поле такі приклади й щоразу натискай Convert to JSON.

Тест A — вкладені елементи

```
<p class="intro">Hello <strong>world</strong>!</p>
```

Очікуваний результат: у p є атрибут class: "intro" і три дочірні вузли за порядком: текст Hello , елемент strong із текстом world, текст !.

Тест B — важливе виправлення SVG

```
<svg><script/><circle cx="1"/></svg>
```

Очікуваний результат: усередині svg містяться два сусідні елементи — script і circle. У circle є атрибут cx: "1". circle не має перетворюватися на текст усередині script.

Тест C — незакритий заголовок

```
<h1>First<h2>Second</h2>
```

Очікуваний результат: h1 із текстом First і h2 із текстом Second — два сусідні елементи в children кореневого документа.

Тест D — HTML-сутності й лапки

```
<p title="a > b">Tom &amp; Jerry &hearts;</p>
```

Очікуваний результат:

```
"title": "a > b"
"value": "Tom & Jerry ♥"
```

Якщо `>` руйнує атрибут або символи не декодуються, це помилка.

Тест E — порожній ввід

Повністю очисти ліве поле й натисни Convert to JSON.

Очікуваний результат:

```
{
"type": "document",
"children": []
}
```

Ці очікувані результати я окремо звірив із функцією з твого ZIP v4, а не просто припустив.

Крок 5. Перевіряємо помилки в Chrome

Після тестів натисни F12. Відкриються інструменти розробника Chrome.

Вибери вкладку Console / Консоль.

Повтори тест SVG із кроку 4 й натисни Convert to JSON.

Що важливо:

- Не має бути червоних повідомлень Uncaught TypeError, ReferenceError, SyntaxError.
- Не має бути помилок завантаження html2json.js.
- JSON має з'являтися праворуч.

Якщо бачиш червону помилку, не виправляй код вручну. Зроби скриншот консолі, щоб можна було встановити точну причину.

Крок 6. Запускаємо автоматичні тести на Windows

Цей крок перевіряє суттєво більше сценаріїв, ніж ручні натискання.

Оскільки в тебе вже використовувалися Node.js та npm для попередніх проєктів, спробуємо використати встановлену версію.

Відкрий розпаковану папку проєкту. Клацни мишкою адресний рядок Провідника Windows, введи:

```
cmd
```

Натисни Enter.

Відкриється командний рядок безпосередньо в папці проєкту.

Спочатку перевір Node.js:

```
node -v
```

Потім npm:

```
npm -v
```

Якщо обидві команди показують номери версій, продовжуй.

Тепер послідовно виконай:

```
npm test
```

Очікуваний результат:

```
# tests 70
# pass 70
# fail 0
```

Час виконання та решта рядків можуть відрізнятися.

Потім:

```
npm run check
```

Якщо команда завершилася без помилки й повернула запрошення командного рядка — синтаксис перевірено.

І останнє:

```
npm run benchmark
```

Цей тест перевіряє продуктивність. Цифри в мілісекундах на твоєму ПК можуть відрізнятися — це нормально. Важливо, щоб команда завершилася без помилок і не зависала.

Жодних `npm install`, `npm audit fix` або змін залежностей зараз не потрібно. Проєкт не використовує сторонніх npm-пакетів.

Крок 7. Що робити, якщо сторінка не відкривається

Якщо за подвійного натискання `index.html` Chrome показує помилку, спробуй відкрити файл безпосередньо через Chrome:

1. Відкрий Chrome.
2. Натисни Ctrl + O.
3. Вибери розпакований index.html.

Якщо й це не працює, можна запустити невеликий локальний сервер. Наприклад, якщо встановлено Python, у командному рядку папки проєкту виконай:

```
py -m http.server 8000
```

Потім відкрий у Chrome:

`http://localhost:8000/index.html`

Не закривай командний рядок, доки перевіряєш сторінку. Якщо Python не встановлений, не потрібно спеціально встановлювати його — просто повідом, яка помилка з'являється.

Підсумкова картка перевірки

Твій прогрес — історичний стан 0 із 7:

- [ ] Архів v4 розпаковано, index.html відкрито у Chrome.
- [ ] Кнопки Input Example 1 та Input Example 2 працюють.
- [ ] Кнопка Convert to JSON показує правильний результат.
- [ ] Тести A–E дають очікувану структуру JSON.
- [ ] У Chrome Console немає помилок застосунку.
- [ ] npm test: 70 пройдено, 0 помилок.
- [ ] npm run check і npm run benchmark завершилися успішно.

Аміго, пропоную спочатку виконати кроки 1–5, тобто ручну перевірку в Chrome. Надішли мені скриншот сторінки з результатом тесту SVG і, якщо є, скриншот помилок консолі. Потім можна надіслати результат npm test.

Не завантажуй поки нічого на GitHub і не надсилай форму jito.dev. Після успішної перевірки залишиться правильно оформити репозиторій та історію AI-розроблення — це окремий, важливий етап.

R276 · AI

Аміго, я переглянув усі 20 скриншотів твоєї ручної перевірки в Chrome. Результат позитивний! Ти правильно пройшов майже всю перевірку інтерфейсу, і на наданих зображеннях я не бачу помилок у роботі конвертера.

Результати перевірки на твоєму комп'ютері

Перевірка	Результат
Input Example 1	PASS
Input Example 2	PASS
Convert to JSON	PASS
Тест A — вкладені елементи	PASS
Тест B — SVG	PASS
Тест C — незакритий заголовок	PASS
Тест D — сутності й лапки	PASS
Тест E — порожній ввід	PASS
Chrome Console	Червоних помилок не видно

Що особливо важливо

1. Виправлення SVG працює. На скриншоті два сусідні елементи script і circle всередині svg. Раніше саме тут був дефект.
2. Неправильна вкладеність заголовків обробляється коректно. h1 і h2 представлені окремими елементами.
3. HTML-сутності декодуються. У результаті тесту D видно Tom & Jerry ♥, а атрибут title зберігає значення a > b.
4. Порожній HTML повертає правильну структуру:

```
{
"type": "document",
"children": []
}
```

5. Приклади компанії перетворюються на JSON. На перших скриншотах після натискання Input Example спочатку відображаються підказки Comment 1 та Comment 2. Це нормальна поведінка вихідного інтерфейсу. Після натискання Convert to JSON з'являється справжнє дерево.

Довгий JSON, прокручування й велика кількість текстових вузлів із пробілами також не є помилкою: вибрана реалізація зберігає вихідні пробіли й переводи рядків.

Залишився один технічний крок на твоєму ПК

Тепер потрібно запустити автоматичні тести. На надісланих скриншотах їхніх результатів поки немає.

Відкрий розпаковану папку v4 у Провіднику Windows. В адресному рядку напиши cmd, натисни Enter.

Послідовно виконай:

```
node -v
npm test
npm run check
npm run benchmark
```

Очікуємо побачити:

Автоматичні тести:

```
# tests 70
# pass 70
# fail 0
```

Кількість тестів стосується версії v4. Точне оформлення виводу може відрізнятися.

Якщо всі команди завершуються без помилок, це підтвердить, що вихідні файли й тести працюють не лише в моєму середовищі, а й на твоєму Windows-ПК.

Що буде після цього

Ми переходимо до фінальної публікації на GitHub, без нових змін алгоритму, якщо не виявиться конкретний дефект.

Залишиться правильно оформити публічний репозиторій, заповнити `ai_help/chatgpt_chat.txt` справжнім посиланням на історію AI-розроблення, перевірити доступність матеріалів в інкогніто й лише потім надіслати результат через портал jito.dev.

Мій вердикт зараз: ручну перевірку інтерфейсу в Chrome пройдено за всіма представленими сценаріями. Це вже серйозний результат. Не потрібно знову тестувати ті самі кнопки.

Аміго, наступна дія — виконай чотири команди вище й надішли скриншот командного рядка з результатами. Якщо там усе зелене, рухаємося безпосередньо до підготовки репозиторію та здачі.

R278 · AI

Аміго, чудово! Тепер я можу підтвердити: версія v4 успішно пройшла автоматичні перевірки безпосередньо на твоєму Windows-комп'ютері. Я уважно вивчив обидва скриншоти командного рядка.

Результати перевірки

Перевірка	Результат
Автоматичні тести	70/70 PASS
Помилки тестів	0
npm run check	PASS
npm run benchmark	Завершений
Ручна перевірка Chrome	PASS

Продуктивність також хороша

Для проблемного випадку з незавершеними HTML-атрибутами твій комп'ютер показав:

Розмір HTML	Медіанний час
10 000 символів	0,091 мс
45 000 символів	0,349 мс
90 000 символів	0,663 мс
360 000 символів	2,456 мс

Це підтверджує, що попереднє суттєве сповільнення усунено. Тести на незавершені закривальні теги спеціальних текстових елементів також завершилися успішно.

Твоя встановлена версія Node.js v23.11.0 впоралася з цими перевірками. Змінювати її заради цього завдання зараз не потрібно.

Що це означає для нашого проєкту

Локальну технічну перевірку завершено. Ми вже підтвердили:

- Роботу інтерфейсу у справжньому Chrome.
- Правильні результати на ручних HTML-прикладах.
- Усі 70 автоматичних тестів.
- Відсутність помилок синтаксису.
- Стійку продуктивність на перевірених складних вхідних даних.

Це достатньо вагома підстава припинити нескінченні доопрацювання й переходити до публікації. Приховані тести роботодавця нам невідомі, тому абсолютної гарантії дати не можна.

Тепер переходимо до GitHub

Пропоную діяти послідовно, щоб випадково не надіслати неправильний архів.

Етап 1. Перевірка коду

Завершений.

Етап 2. Підготовка репозиторію GitHub

Створюємо окремий публічний репозиторій і завантажуємо перевірені файли v4.

Етап 3. Історія AI

Додаємо справжнє посилання, перевіряємо доступ і дотримання вимоги компанії про повну історію використання AI. Окремо потрібно коректно вирішити питання попереднього технічного аудиту в цьому приватному чаті.

Етап 4. Фінальна перевірка й здача

Перевіряємо вміст репозиторію та посилання в інкогніто. Лише після цього заповнюємо форму jito.dev.

Твій наступний крок — створити репозиторій

1. Відкрий Створення нового GitHub-репозиторію.
2. У полі Repository name вкажи `jito-html2json-test-task`.
3. Вибери Public.
4. Не вмикай автоматичне створення README, .gitignore або ліцензії — у нас уже є підготовлені файли.
5. Натисни Create repository.

Поки не завантажуй файлів і не надсилай посилання jito.dev. Спочатку правильно підготуємо `ai_help/chatgpt_chat.txt`, щоб репозиторій одразу містив повний комплект матеріалів.

Аміго, технічна частина вже виглядає добре. Тепер головне — акуратно пройти останні організаційні кроки.

R296 · AI

Аміго, я знайшов одне конкретне недопрацювання в останньому архіві, тому добре, що ти попросив перевірити все перед публікацією.

У README.md досі написано, що файл ai_help/chatgpt_chat.txt містить заглушку, хоча у твоєму архіві вже є справжнє посилання на робочий чат. Якщо залишити так, сеньйор-розробник побачить суперечність між документацією та файлами.

Це легко виправити. Самого коду переписувати не потрібно.

Що вже підтверджено під час повторної перевірки останнього архіву:

- 70 із 70 тестів проходять у Node.js 22.16.0.
- npm run check завершується успішно.
- npm run benchmark виконується без помилки.
- На місці html2json.js, вихідний index.html, 31 HTML-приклад, тести й папка ai_help.
- GitHub-репозиторій flankero-dev/jito-html2json-test-task існує, публічний і поки порожній.

Я також повторно звірив комплект з офіційним README компанії. Обов'язкової структури дотримано.

Я виправив недопрацювання й підготував остаточний архів

`jito-html2json-final-for-github.zip`

38 файлів · 27 КБ · готовий до завантаження в GitHub.

Завантажити остаточний архів.

У ньому я виправив застарілі відомості в README, зазначив фактично виконану ручну перевірку Chrome й залишив у ai_help/chatgpt_chat.txt лише назву основної AI-переписки та її посилання.

html2json.js, тести, HTML-приклади й вихідний index.html не змінено. Після підготовки архіву я знову отримав 70/70 PASS та успішну перевірку синтаксису.

Станом на цей момент відомих технічних блокерів для публікації немає. Але гарантувати проходження прихованих тестів не можна. Залишається й формальне питання повноти AI-історії: компанія просить усю історію, а окремі перевірки проводилися в цьому чаті. Це потрібно враховувати до остаточного надсилання роботодавцю.

Докладна інструкція із завантаження в GitHub

1

Завантаж архів вище.

У папці «Завантаження» Windows натисни на нього правою кнопкою → Видобути все → Видобути.

2

Відкрий розпаковану папку.

Усередині має бути рівно вісім елементів верхнього рівня:

```
README.md
html2json.js
index.html
package.json
ai_help/
benchmarks/
html_samples/
tests/
```

Не має бути додаткової вкладеної папки з назвою проєкту.

3

Відкрий свій порожній GitHub-репозиторій:

`flankero-dev/jito-html2json-test-task`

На сторінці Quick setup натисни посилання uploading an existing file.

4

Завантаж вміст проєкту.

Повернися у Провідник Windows, перебуваючи всередині розпакованої папки.

Натисни Ctrl + A, щоб виділити всі вісім елементів. Перетягни виділене мишкою в область завантаження на GitHub.

Не перетягуй самого ZIP.

5

Перевір завантажені шляхи.

GitHub має показувати файли, зокрема:

```
html2json.js
index.html
README.md
package.json
ai_help/chatgpt_chat.txt
tests/html2json.test.js
benchmarks/malformed-input.js
html_samples/01-full-document.html
...
html_samples/31-svg-html-integration-point.html
```

Особливо важливо: усі файли мають опинитися в корені репозиторію або своїх правильних папках, а не всередині додаткового каталогу.

6

Підготуй коміт.

У полі Commit message можеш зазначити:

```
Add HTML to JSON converter with tests and examples
```

Залиш варіант Commit directly to the main branch.

7

Натисни Commit changes, але лише коли переконаєшся, що всі файли й папки відображаються правильно.

Після завершення GitHub відкриє сторінку репозиторію з README та файлами проєкту.

8

Перевірка після публікації.

Відкрий репозиторій в інкогніто. Перевір, що доступні html2json.js, index.html, папка html_samples та файл ai_help/chatgpt_chat.txt. Переконайся, що посилання з останнього файла відкриває робочий чат.

Аміго, зараз виконай кроки 1–5 й надішли скриншот GitHub зі списком файлів перед натисканням Commit changes. Я перевірю розташування файлів, щоб ми не зіпсували коректно підготовленого комплекту.

Після публікації перевіримо GitHub-репозиторій уже ззовні. Форму jito.dev поки не надсилаємо.

R300 · AI

Аміго, я ще раз незалежно перевірив останній архів, офіційний README jito.dev, вихідний код і тести. І тепер можу дати остаточну оцінку.

За технічною частиною ми виконали завдання якісно. Але сказати, що абсолютно все готове до здачі й ризику відмови немає, я не можу. Залишився один суттєвий формальний нюанс.

1. Звірення з вимогами jito.dev

```
| Вимога компанії | Результат |
|---|---|
| Реалізувати html2json(htmlText) | Виконано |
| Перетворювати HTML на JSON-структуру | Виконано |
| Не використовувати DOM-парсер | Перевірено |
| Зберегти index.html | Виконано |
| Створити html_samples/ | 31 файл |
| Обробляти різні структури HTML | Перевірено |
| Не допускати падінь на нестандартному вводі | На перевірених випадках |
| Забезпечити читабельність коду | Перевірено |
| Використовувати AI | Виконано |
| Додати ai_help/chatgpt_chat.txt | Файл є |
| Надати повну історію AI | Ще не завантажений |
| Здати посилання через портал |
```

Примітка укладача: отриманий текст цього блоку закінчується на незавершеному рядку. Відсутнього висновку або статусу не дописано.

R306 · AI

Аміго, незалежний аудит ChatGPT Work виявив справжні помилки, які наші 70 тестів пропустили. Тому поки нічого не завантажуємо в GitHub.

У звіті підсумковий статус — NOT READY. Водночас ситуація виправна: Work не пропонує переписувати весь парсер, а вказує на чотири конкретні проблеми.

Що виявлено

Проблема	Пріоритет	Суть
D1 — Unicode	P1, високий	Символ İ порушує розрахунок позицій, через що парсер втрачає наступні HTML-елементи
D2 — SVG-атрибути	P2, середній	Атрибути constructor та proto перетворюються на неправильні імена
D3 — імена тегів	P2, середній	`<my-widget.v2>` розбирається неправильно
D4 — посилання README	P1, високий	Markdown-посилання на історію ChatGPT може вести за неправильною адресою

Особливо важливий D1: функція не падає, але мовчки повертає неправильний JSON. Для завдання jito.dev, де оцінюють незвичайні вхідні дані, це суттєвий дефект.

Незалежна перевірка також показала 70/70 PASS на старих тестах, але лише 30/35 PASS на нових структурних перевірках. Це пояснює, чому попередні запевнення про повну технічну готовність були передчасними.

Я перевірив висновки Work на останньому архіві

І тут є важливе уточнення: три помилки в коді підтвердилися незалежно й у нашому останньому архіві

`jito-html2json-clean-readme-for-github.zip`.

Я сам запустив п'ять проблемних прикладів через Node.js 22.16.0. Результати збіглися з описом Work: Unicode порушує розбір, SVG-атрибути пошкоджуються, а ім'я my-widget.v2 обрізається.

Водночас старі 70/70 тестів як і раніше проходять. Отже, потрібні нові регресійні тести.

Але четверту помилку вже виправлено

Work виявив пошкоджене Markdown-посилання в іншому архіві. Я перевірив наш останній скорочений README: у ньому посилання коректне.

```
The ChatGPT development conversation is linked in [`ai_help/chatgpt_chat.txt`](ai_help/chatgpt_chat.txt).
```

Крім того, звіт Work прив'язаний до певної версії ZIP за SHA-256, що відрізняється від останнього підготовленого архіву. Ми перевірили не лише збіг назв: html2json.js у цих двох доступних архівах побайтово однаковий. Тому D1–D3 стосуються поточного коду, а D4 — ні.

Що надсилаємо в український робочий чат

<summary>Повний український оригінал створеного промпта</summary>

Перед публікацією вирішив зробити ще одну незалежну перевірку. Вона виявила три конкретні помилки, які наші поточні тести не помітили.

Перевір, будь ласка, такі випадки:

1. Unicode та позиції в тексті

```
İ<script>x</script><p>y</p>
```

Тут після script має залишитися окремий елемент p, але зараз його вміст потрапляє всередину script.

Схожа проблема є з:

```
<textarea>İ</textarea><p>y</p>
```

А також із розпізнаванням DOCTYPE після символу İ.

Схоже, причина в input.toLowerCase(): деякі Unicode-символи змінюють довжину після приведення до нижнього регістру, через що зсуваються індекси.

2. SVG-атрибути

```
<svg constructor="x" __proto__="y"/>
```

Замість збереження обох назв атрибутів парсер підставляє значення успадкованих властивостей JavaScript-об'єкта. Перевір таблицю SVG_ATTRIBUTE_CASE і використання Object.hasOwn().

3. Імена custom elements

```
<my-widget.v2>x</my-widget.v2>
```

Зараз парсер обрізає назву до my-widget, а решту розмітки обробляє неправильно. Потрібно перевірити правила читання імен початкових і кінцевих тегів.

Прошу відтворити всі три проблеми, виправити їх мінімальними змінами та додати точні регресійні тести. Не потрібно переписувати архітектуру чи розширювати проєкт до повного HTML5-парсера.

Окремо перевір посилання в README. У нашій останній скороченій версії воно має вести на ai_help/chatgpt_chat.txt. Якщо воно вже правильне, не змінюй його без потреби.

Після виправлень повторно запусти всі тести, перевір синтаксис і продуктивність, а також переконайся, що попередні SVG/MathML-виправлення не зламалися.

Надай оновлений ZIP і короткий звіт із фактичними результатами. Поки нічого не публікуємо в GitHub і не відправляємо роботодавцю.

Наш поточний статус

- 70 наявних тестів проходять.
- Посилання README в останньому архіві коректне.
- Три відтворювані дефекти потребують виправлення.
- GitHub-публікацію й здачу поки призупинено.

Аміго, незалежний аудит виявився справді корисним. Він виявив не косметичні недоліки, а помилки, що призводять до неправильного JSON.

І я повинен визнати: мої попередні запевнення, що технічна частина повністю готова, були передчасними.

Тепер діємо точково: виправляємо три причини, підтверджуємо їх тестами й знову перевіряємо фінальний архів. Після цього повертаємося до публікації, не починаючи весь процес заново.

Питання повної AI-історії також залишається окремим формальним ризиком, який потрібно вирішити до надсилання роботодавцю.

R310 · AI

Аміго, результат ChatGPT Work суттєво кращий за попередній, але статус поки не «повністю готово» — READY WITH RESERVATIONS.

Я вивчив прикріплений технічний звіт, а не лише скриншот.

Перевірка / показник	Результат
Виправлення D1–D4	4/4
Автоматичні тести	85/85 PASS
Незалежні структурні сценарії	35/35 PASS
Генеровані дерева	3 000 PASS
Некоректні входи	15 000 перевірок
Справжній Chrome на новій версії	Не перевірений

Особливо цінно, що Work повідомив не лише про успішні тести, а й про перевірку нових регресійних тестів на старому коді: 13 із 15 справді виявляли помилки до виправлень. Це набагато переконливіше за просте збільшення кількості тестів із 70 до 85.

Однак це поки відомості зі звіту Work, а не результати мого самостійного запуску нового фінального ZIP.

Що я додатково помітив у звіті

Є три важливі моменти.

1. Work використав інший вихідний архів, ніж ми спочатку планували. У звіті зазначено `jito-html2json-task_особисте ім’я вилучено [1].zip`. Водночас Work зафіксував його SHA-256 і докладно перелічив внесені зміни. Це не обов'язково проблема, але означає, що орієнтуватися тепер потрібно виключно на новий підсумковий архів, а не на наші старі версії.
2. До фінального README додано фразу про додаткові AI-сесії. Це відповідає прагненню чесно відобразити процес, але ти раніше просив залишити оформлення максимально лаконічним. Перед публікацією варто прочитати цей рядок самому. Видаляти відомості лише заради приховування використання AI не рекомендую.
3. Work не запускав нового коду у справжньому Chrome. Попередню версію v4 ти вже перевірив вручну, але після неї змінився html2json.js, тому потрібен короткий повторний браузерний тест.

Наступний крок

Завантаж із ChatGPT Work файл `jito-html2json-verified-final.zip` і прикріпи його сюди.

Зараз у мене є повний технічний звіт, але самого нового ZIP серед доступних файлів немає. Тому я не заявлятиму, що самостійно перевірив його вміст.

Очікуваний SHA-256 архіву згідно зі звітом:

`a75dd83892033d986aa84a7f29249c8f63ab2688a98ad9cee4944317adb2bf41`

Після отримання ZIP перевіримо фактичні файли й тести. Потім на твоєму ПК залишиться коротка перевірка у Chrome:

HTML для вводу	Що має вийти
`İ<script>x</script><p>y</p>`	Текст İ, окремі script та p
`<svg constructor="x" proto="y"/>`	Обидва атрибути з правильними назвами
`<my-widget.v2>x</my-widget.v2>`	Елемент з ім'ям my-widget.v2

І перевіримо, що в Console немає помилок.

Мій поточний висновок: Work підготував переконливий звіт про технічно виправлену версію. Це підстава перейти до заключної перевірки, а не починати черговий масштабний аудит.

Але публікацію GitHub і надсилання jito.dev я поки не вважаю завершеними. Спочатку підтвердимо новий ZIP, потім перевіримо Chrome й окремо вирішимо формальну вимогу щодо повної історії AI-розроблення.

R312 · AI

Аміго, цей SHA-256 точно збігається з контрольною сумою, зазначеною у фінальному звіті ChatGPT Work.

Очікуваний SHA-256 фінального архіву:

`a75dd83892033d986aa84a7f29249c8f63ab2688a98ad9cee4944317adb2bf41`

Збігається зі значенням у звіті Work.

Але збіг тексту зі звітом ще не доводить цілісності самого ZIP.

Що робимо далі

1. Завантаж із ChatGPT Work файл jito-html2json-verified-final.zip.
2. Прикріпи сюди сам ZIP, а не лише його SHA-256.
3. Я перевірю контрольну суму архіву, файли, README й самостійно запущу 85 тестів.

Після цього залишиться перевірити виправлені випадки у Chrome на твоєму ПК й завантажити остаточний комплект у GitHub.

Старого архіву із 70 тестами більше не використовуємо. Нового поки не публікуємо.

R314 · AI

Аміго, отримав фінальний ZIP від ChatGPT Work і самостійно перевірив його вміст. Цього разу перевірку виконано безпосередньо на завантаженому файлі, а не лише за звітом.

- SHA-256 — збігається.
- Файлів у ZIP — 39.
- npm test — 85/85 PASS.
- npm run check.
- npm run benchmark.

Перевірена контрольна сума:

`a75dd83892033d986aa84a7f29249c8f63ab2688a98ad9cee4944317adb2bf41`

Усі команди завершилися успішно на Node.js 22.16.0. README містить коректне Markdown-посилання, 85 тестів описано правильно, а нові регресійні тести присутні в архіві.

Це вже незалежне підтвердження ключових результатів Work. Залишилося оцінити саме виправлені випадки D1–D3 й переконатися, що жодні останні зміни не потребують повторної перевірки у Chrome.

Підсумок незалежної перевірки

Версія jito-html2json-verified-final.zip пройшла мої технічні перевірки. Виправлення D1–D4 присутні, а нових помилок у перевірених сценаріях я не виявив.

Окрім 85 штатних тестів, я перевірив вісім додаткових випадків через Node.js та 11 сценаріїв у Chromium 144. У браузері не було помилок Console; введений HTML не виконувався як JavaScript.

Також підтверджено, що:

- Оригінальний index.html не змінений.
- README.md коректно оформлений, без зламаних Markdown-посилань.
- У ZIP рівно 39 файлів, без зайвої зовнішньої папки.
- html2json.js не використовує заборонених DOM-парсерів.
- ai_help/chatgpt_chat.txt містить посилання на основний робочий чат.

Технічний вердикт: готовий до заключної перевірки на Windows і публікації. Невідомі тести роботодавця та вимога повної AI-історії залишаються окремими ризиками.

Що зробити на твоєму ПК

Розпакуй новий jito-html2json-verified-final.zip, а не стару версію v4.

Відкрий index.html у Chrome.

По черзі встав три приклади нижче й натисни Convert to JSON:

```
İ<script>x</script><p>y</p>
```

Очікується текст İ та два окремі елементи script і p.

```
<svg constructor="x" __proto__="y"/>
```

Очікуються атрибути `"constructor": "x"` та `"proto": "y"`.

```
<my-widget.v2>x</my-widget.v2>
```

Очікується `"tagName": "my-widget.v2"`.

Потім відкрий командний рядок у папці проєкту й виконай:

```
npm test
npm run check
npm run benchmark
```

Очікуваний підсумок npm test — 85 пройдених тестів, 0 помилок.

Після цього можемо завантажувати вміст цього ZIP у твій уже створений репозиторій jito-html2json-test-task. Самого ZIP та окремого аудиторського звіту завантажувати не потрібно.

Важливо: перед остаточним надсиланням jito.dev ще потрібно буде вирішити питання повноти історії використання AI. Зараз в архіві є посилання лише на основний чат, тоді як додаткові перевірки й виправлення виконувалися в інших сесіях.

Аміго, цього разу в нас є не просто позитивний звіт Work, а контрольна сума, що збігається, самостійно запущені тести й незалежна перевірка виправлень. Тепер головне — підтвердити нову версію на твоїй Windows і правильно завершити публікацію.

R320 · AI

Аміго, вивчив три файли останнього аудиту Work і перевірив сам ZIP. Хороша новина: Work не виявив нових критичних дефектів P0/P1 і не став переписувати працюючий код.

Я також самостійно повторив перевірки на Node.js 22.16.0 та в Chromium.

Перевірка	Результат
SHA-256 фінального ZIP	Збігається
Автоматичні тести	85/85 PASS
Синтаксис і benchmark	PASS
Файлів в архіві	39
Браузерні сценарії	Без помилок JS

У Chromium я перевірив обидві кнопки прикладів, конвертацію та виправлення Unicode, SVG й імен тегів. Застосунок працює. Уточнення: браузер запускався з окремо завантаженими вихідними HTML і JavaScript — пряме відкриття file:// блокується моїм середовищем.

Що знайшов senior-аудит

P2 — F-01. SVG із CDATA

```
<svg><![CDATA[a<b>c]]></svg>
```

Замість одного текстового вузла конвертер створює коментар і залишковий текст.

P2 — F-02. Вкладеність p усередині foreignObject

```
<p><svg><foreignObject><p>x</p></foreignObject></svg>
```

Внутрішній p помилково переноситься на верхній рівень дерева.

Я підтвердив обидві відмінності порівнянням зі справжнім Chromium. Вони існують, і senior справді може їх помітити. Але це складні випадки SVG/HTML-взаємодії, а поточний README не обіцяє повної HTML5-сумісності.

Work класифікував їх як P2 і рекомендував не вносити нових змін без потреби, оскільки спроба реалізувати повні HTML5-правила може призвести до регресій. Я вважаю це розумним рішенням для нинішнього етапу.

Три вердикти Work

Напрям	Результат
Технічна готовність	READY WITH RESERVATIONS
Готовність до надсилання	NOT READY
Готовність до code review	READY WITH RESERVATIONS

Це відповідає змісту звіту: два обмеження залишаються, повної історії AI ще не надано, а твоє фактичне розуміння коду поки не перевірялося на співбесіді. [Посилання оригіналу: `JITO_SENIOR_REVIEW_FINAL_REPORT.md`.]

Зверни увагу: P2 — оцінка Work, а не погоджений із jito.dev ступінь серйозності помилок. Senior може оцінити їх суворіше. Однак черговий масштабний рефакторинг без конкретної потреби також ризикований.

Що робити тепер

1. Не змінювати html2json.js на підставі цього аудиту. Work не вніс змін і зберіг той самий релізний ZIP із SHA-256 `a75dd838...adb2bf41`.
2. Перевірити цю версію на Windows. Запустити npm test (85/85), відкрити index.html у Chrome, перевірити обидві кнопки прикладів, Unicode, SVG і my-widget.v2. Моя перевірка в Chromium уже показала правильну роботу, але пряме відкриття файлів у моєму середовищі заблоковане.
3. Закрити питання AI-історії. Основний публічний чат не включає всіх додаткових сесій. Потрібно підготувати історію, що стосується завдання, і погодити спосіб надання приватних матеріалів, якщо вони містять сторонні особисті дані.
4. Завантажити фінальні 39 файлів у GitHub, перевірити README та всі публічні посилання в інкогніто. Твій репозиторій я перевірив: він поки публічний, але порожній.
5. Підготуватися до технічного захисту. Не публікуй JITO_CODE_REVIEW_PREPARATION_PRIVATE.md: це виключно навчальний матеріал. Почати варто з токенізатора, стека, yield, asciiLower та Object.hasOwn. [Посилання оригіналу: `JITO_CODE_REVIEW_PREPARATION_PRIVATE.md`.]

Я додатково перевірив у справжньому Chromium, що HTML зі script та обробником onerror, вставлений у поле вводу, не виконується під час конвертації.

Моя рекомендація: більше не замовляти масштабних AI-аудитів. Ми отримали достатньо доказів, щоб завершувати технічну роботу, хоча абсолютної гарантії відсутності помилок немає. Зараз головне завдання — забезпечити коректну здачу й твою здатність пояснити рішення.

Наступною дією я пропоную перевірити останній ZIP на твоїй Windows, а потім зайнятися GitHub. Після цього залишиться вирішити формальне питання AI-історії до надсилання роботодавцю.

R326 · AI

Аміго, нам потрібен не просто «вау-ефект», а рішення, у якому сеньйор не знайде очевидних проблем, і кандидат, здатний захистити кожне важливе рішення. Гарантувати проходження етапу неможливо, але суттєво зменшити ризик — можна.

Я почав додаткову перевірку проти справжнього Chromium і отримав конкретний результат: із 91 спеціально

підібраного сценарію 67 дали дерева, що точно збігаються, а 24 — відмінності. Більшість відмінностей стосуються вже відомих обмежень: HTML-сутності, відновлення вкладеності, SVG/MathML.

Але виявилася нова проблема, яку попередні аудити не виділяли. На двох коротких некоректних HTML-входах наш парсер поглинає всі наступні елементи:

```
<!--><p>ok</p>
```

```
<!---><p>ok</p>
```

Chromium в обох випадках створює порожній коментар і зберігає окремий елемент `p`. Наша поточна версія перетворює практично весь ввід на коментар, втрачаючи `p`.

Це саме той тип неочікуваної ситуації, який варто перевірити перед здачею. Я перевірю можливість невеликого ізольованого виправлення, що не зачіпає решту алгоритму.

Виправлення вдалося ізолювати

Я підготував окрему посилену копію проєкту, не змінюючи попередній перевірений ZIP. Змінено лише механізм завершення HTML-коментарів; додано 10 регресійних тестів.

Результати:

- На старій реалізації нові тести: 5 PASS / 5 FAIL.
- На виправленій реалізації: 95 PASS / 0 FAIL.
- `npm run check` і `npm run benchmark`: PASS.
- Нові варіанти коментарів звірено з Chromium: збіги підтверджено.

Це не спроба реалізувати весь HTML5. Ми усунули конкретний клас помилок, який призводив до мовчазної втрати елементів після некоректного коментаря.

Наступний контроль — відкрити саме виправлену копію як звичайну HTML-сторінку в Chromium, а потім повторно перевірити ZIP після пакування.

Нова посилена версія готова

Я зібрав архів із виправленням, повторно розпакував його в чисту папку й запустив усі штатні перевірки саме звідти.

Перевірка / показник	Результат
Автоматичні тести	95/95 PASS
Синтаксис JavaScript	PASS
Benchmark	PASS
Порівняння нових випадків із Chromium	10/10
Файлів в архіві	40

Завантажити посилений HTML-to-JSON проєкт

Завантажити технічний звіт із доказами

Примітка укладача про вкладення: попередні два рядки — переклад підписів посилань оригінального повідомлення. Їхні адреси не підмінено публічними URL. У наданому handoff є `jito-html2json-hardened-candidate.zip` із наведеним нижче SHA-256 та `JITO_TARGETED_ADVERSARIAL_HARDENING_REPORT.md`; це відповідні збережені артефакти. Ідентичність архіву встановлюється за повним хешем, а не лише за назвою.

SHA-256 нового ZIP: `d46cfe82f3e77cf761d9d496a56c9f936c0df66c4fe845e13bf28b63b0783a8f`

Наскільки це підвищує наші шанси?

Ми закрили ще один реальний клас помилок, але не перетворили рішення на бездоганний HTML5-парсер. Із 91 спеціально вибраного браузерного сценарію точні збіги зросли з 67 до 69; решта відмінностей потребують

окремої оцінки й значною мірою стосуються відомих обмежень.

Я рекомендую розглядати новий ZIP із 95 тестами як основний кандидат на здачу, оскільки зміна локальна, попередні тести не зачеплено, а нові демонструють помилку до виправлення та її відсутність після.

Але для справжнього «вау-ефекту» потрібні ще три речі: акуратний GitHub, повна й доступна AI-історія та впевнений технічний захист. Найбільш вражаючий результат для сеньйора — не кількість тестів, а здатність показати, яку помилку знайшли, як відтворили, чому виправлення правильне та які обмеження свідомо залишили.

Наступний пріоритет — відкрити саме цей новий ZIP на твоїй Windows, перевірити інтерфейс у Chrome, а потім завершити GitHub та AI-історію. Публікацію й надсилання роботодавцю я не виконував.

Ми вже знайшли дефект, який пропустили попередні аудити, й усунули його з перевірними доказами. Продовжувати можна, але обіцяти, що тепер абсолютно всі можливі нестандартні випадки передбачено, було б неправильно.

R330 · AI

Ось архів, який потрібно прикріпити до ChatGPT Work:

Завантажити `jito-html2json-hardened-candidate.zip`

29 КБ · 95 автоматичних тестів · 40 файлів

SHA-256 перевірено, архів доступний:

`d46cfe82f3e77cf761d9d496a56c9f936c0df66c4fe845e13bf28b63b0783a8f`

Додатково приклади до Work технічний звіт:

Завантажити `JITO_TARGETED_ADVERSARIAL_HARDENING_REPORT.md`

Використовуй саме цей ZIP як вихідну версію для останнього інженерного етапу.

R332 · AI

На скриншоті бачу помилку «Не удалось загрузить этот файл» під час відкриття звіту `JITO_TARGETED_ADVERSARIAL_HARDENING_REPORT.md`.

Я перевірив: файл існує, вміст не пошкоджено. Схоже, проблема саме у вбудованому переглядачі ChatGPT.

Я підготував дві альтернативи.

1. Звіт у ZIP — рекомендую цей варіант:

Завантажити `JITO_TARGETED_ADVERSARIAL_HARDENING_REPORT.zip`

2. Звіт у звичайному текстовому форматі:

Завантажити звіт `.txt`

А сам проєкт із 95 тестами тут:

Завантажити `jito-html2json-hardened-candidate.zip`

Для ChatGPT Work: прикріпи архів проєкту й звіт у форматі `.txt`. Їхній вміст я перевірив; звіт у TXT повністю ідентичний Markdown-оригіналу.

Якщо після натискання відкривається переглядач із помилкою, спробуй значок у правому верхньому куті — він завантажує файл без потреби переглядати його всередині ChatGPT.

R334 · AI

Аміго, новий архів ChatGPT Work отримано, і його основні технічні результати я вже підтвердив самостійно. Це справді нова версія з виправленнями, а не перейменований старий ZIP.

Перевірка / показник	Результат
SHA-256	Збігається
Склад архіву	41 файл
`npm test`	128/128 PASS
`npm run check`	PASS
`npm run benchmark`	PASS

Перевіряв не за текстом звіту, а з чистого розпакування завантаженого ZIP на Node.js 22.16.0.

Контрольна сума:

`bd513a2d4b0a152bfc6f0f948664983bd4e4d5841041cd2a09640f7c736cf6b3`

Це підтверджує, що Work передав коректно зібраний технічний комплект. Але зелені тести ще не означають, що всі можливі HTML-конструкції обробляються як у Chrome.

Що змінилося порівняно з попередньою версією

Останній Work справді усунув кілька недоліків, заради яких і проводилася додаткова інженерна робота.

Виправлення	Що стало краще
E1 — повільні коментарі	Усунено повторне сканування решти документа, яке давало квадратичний час
E2 — некоректні декларації	Конструкції з лапками більше не поглинають наступні HTML-елементи
E3 — коментарі наприкінці файла	Виправлено обробку незавершених роздільників
E4 — SVG/MathML CDATA	Вміст розпізнається як буквальний текст у підтримуваному foreign-контексті
E5 — вкладені p	Виправлено область дії закриття p на деяких межах HTML/SVG/MathML
E6 — великі рядки	Зменшено тимчасові витрати пам'яті під час ASCII-приведення регістру
E7 — запуск тестів	Прибрано залежність від розкриття `*.test.js` командною оболонкою

Найважливіший новий дефект — E1: на великій кількості звичайних коректних HTML-коментарів стара версія різко сповільнювалася. Work показав вимірювання до/після, і мій незалежний benchmark підтверджує, що нова версія швидко обробляє великі серії коментарів. [Посилання оригіналу на `JITO_ENGINEERING_FINAL_RELEASE_REPORT.md`.]

Додаткова браузерна перевірка

Я окремо перевірив обидві кнопки прикладів і 12 інших сценаріїв у Chromium 144. Усі 14 перевірок пройшли, помилок JavaScript не виникло.

Є застереження: моє середовище блокує відкриття `file://` та звичайну мережеву навігацію. Тому в моїй перевірці HTML і JavaScript завантажувалися окремо в браузерний тестовий контекст. Це не повноцінна перевірка відкриття

файла на Windows.

Сам Work повідомляє, що перевірив вихідний `index.html` через справжній `file://` у Chromium 153: 18/18 PASS. Це твердження підтверджено його звітом, але не моїм повторним запуском у такому режимі. [Посилання оригіналу на `JITO_ENGINEERING_FINAL_RELEASE_REPORT.md`.]

Що ще не можна вважати ідеальним

Новий звіт чесно фіксує 40 відмінностей на 143 порівняннях із Chromium. Серед них — неповна підтримка HTML-сутностей, складних таблиць, відновлення неправильної вкладеності й деяких станів `script`. Це не означає, що 40 обов'язкових тестів роботодавця провалено: порівнювалися можливості, частина яких виходить за заявлений контракт. [Посилання оригіналу на `JITO_ENGINEERING_FINAL_RELEASE_REPORT.md`.]

Але й гарантії проходження всіх прихованих тестів це не дає.

Поточний стан

Напрям	Вердикт
Технічна реалізація	READY WITH RESERVATIONS
Автоматичні перевірки	128/128 PASS
Готовність до GitHub	Код готовий до публікації
Готовність до надсилання jito.dev	NOT READY
Захист рішення перед senior	Потребує твоєї практичної підготовки

Головний формальний блокер не змінився: файл `ai_help/chatgpt_chat.txt` як і раніше містить лише основне ChatGPT-посилання. Повну історію додаткових AI-сесій не оформлено. [Посилання оригіналу на `JITO_ENGINEERING_FINAL_RELEASE_REPORT.md`.]

Твій GitHub-репозиторій я також перевірив: він публічний, але поки порожній.

Які файли використовувати тепер

`jito-html2json-engineering-final.zip`

Єдиний актуальний кандидат для публікації. 41 файл, 128 тестів.

`JITO_ENGINEERING_FINAL_RELEASE_REPORT.md`

Докладний технічний звіт Work. У GitHub без потреби не завантажувати.

`JITO_SENIOR_DEFENSE_PREPARATION.md`

Приватний навчальний матеріал для технічного захисту. Не публікувати.

Мій висновок: це найсильніша й найбільш ретельно перевірена версія проєкту з усіх, які ми отримували. Я не знайшов нового критичного дефекту у виконаних незалежних перевірках.

Тепер я рекомендую зафіксувати цей ZIP як технічний релізний кандидат, не запускати черговий нескінченний цикл перероблення, а закрити обов'язкові умови здачі: AI-історію, справжню Windows-перевірку, GitHub та підготовку до захисту.

Особливо важливе останнє: Work підготував 15 запитань, 5 практичних завдань із коду й 5 підступних запитань. Але доки ти не можеш самостійно відповісти на них, не можна вважати підготовку до senior-review завершеною.

Код уже достатньо зрілий, щоб переходити до здачі. Саму здачу ще необхідно правильно завершити.

R342 · AI

Аміго, отримав нову фінальну версію від Work і перевірив її безпосередньо за завантаженим архівом `jito-html2json-release-candidate-final.zip`.

Перевірка / показник	Результат
SHA-256	Збігається зі звітом
Цілісність ZIP	PASS
Файли	46
`npm test`	201/201 PASS
`npm run check`	PASS
`npm run benchmark`	PASS

Контрольна сума: `e814416cc0bfb15ff663c09b3c2efcc0682db3df8d894eeff0024e0ddb333fcd`.

Усі три команди успішно виконано мною з чистого розпакування на Node.js 22.16.0. Код справді змінено: тепер він містить повну таблицю іменованих HTML-сутностей, більш розвинену обробку таблиць і станів `script`, додаткові перевірки помилкових атрибутів і тегів.

Це суттєво більш розвинене рішення, ніж вихідна версія із 70 тестами. Однак 201 успішний тест ще не означає абсолютної HTML5-сумісності — зараз окремо перевіряю результати нових виправлень і залишкові ризики перед здачею.

Підсумок перевірки нової версії

Я додатково перевірив інтерфейс у Chromium 144 із завантаженими вихідними HTML і JavaScript. Обидві кнопки прикладів, перетворення, відновлення після некоректної розмітки та захист від виконання введеного JavaScript спрацювали. Помилок у Console не виникло. Це додаткова перевірка, але не заміна прямому відкриттю файлів через `file://`.

За даними звіту Work, результат суттєво поліпшено:

- 201/201 штатних тестів.
- 11 579/11 579 перевірок HTML-сутностей.
- 6 500/6 500 структурних генеративних сценаріїв.
- 25 000/25 000 випробувань стійкості до пошкодженого вводу.
- 24/24 браузерних сценарії через `file://` і стільки ж через HTTP.

Залишилося 27 відмінностей зі 143 порівнянь із Chromium — частина становить свідомі відмінності контрактів, але повної HTML5-сумісності не досягнуто. [Посилання оригіналу на `JITO_FINAL_ACCEPTANCE_EVIDENCE.md`.]

Великі додаткові набори я самостійно повністю не відтворював. Вони підтверджені звітом Work, тоді як 201 штатний тест і 15 точкових сценаріїв я запустив самостійно.

Мій вердикт

Область	Статус
Реалізація та основні тести	Перевірено успішно
Готовність до критичного code review	Висока, із відомими обмеженнями
Повна HTML5-сумісність	Не заявлена
Готовність до надсилання роботодавцю	Поки ні

Залишилося два суттєві організаційні питання: зібрати повну історію використання AI та опублікувати проєкт. Перевірив репозиторій `jito-html2json-test-task` — він досі порожній.

І ще: новий `html2json.js` займає близько 1300 рядків. Тому на senior-review особливо важливо вміти пояснити не лише загальний стек, а й декодування сутностей, обробку таблиць і нетривіальні стани `script`.

Файли останньої версії

Фінальний ZIP — 201 тест

Єдина актуальна версія для майбутньої публікації

Технічний звіт Work

Докази, виправлення, залишкові обмеження

Підготовка до senior code review

Приватний навчальний документ — у GitHub не завантажувати

Рекомендую зафіксувати цю версію. Нового загального завдання для Work зараз не потрібно. Наступний крок — не ще одне коло виправлень, а повна AI-історія, перевірка на твоїй Windows, публікація на GitHub і захист коду. Саме ці незакриті пункти тепер безпосередньо впливають на можливість успішної здачі.

R350 · AI

Аміго, давай проведемо максимально повноцінну перевірку JITO.DEV безпосередньо на твоїй Windows, так, як це зробив би QA-інженер перед передачею проєкту сеньйору-розробнику.

Я вивчив саме наш останній ZIP із 201 тестом і підготував для тебе автоматичний Windows QA-скрипт. Він перевіряє архів, запускає тести, випробовує нестандартні HTML-конструкції та зберігає результати в окрему папку.

Але однієї автоматичної перевірки недостатньо. Тому ми зробимо два етапи: автоматичний QA + ручний end-to-end тест у Google Chrome.

Що перевірятимемо

- Цілісність вихідного ZIP і файлів.
- 201 автоматичний тест.
- 43 додаткові перевірки, включно з 31 HTML-прикладом.
- Benchmark та агресивні вхідні дані.
- Справжній Chrome: кнопки, JSON, помилки Console.
- Запуск як `file://` і через локальний HTTP.

Етап 1. Підготовка Windows

Крок 1. Завантаж два файли

`jito-html2json-release-candidate-final.zip`

Наш актуальний проєкт — 46 файлів, 201 тест.

`JITO_WINDOWS_QA.ps1`

Створений мною Windows-скрипт автоматичної перевірки.

Створи на робочому столі папку `JITO-QA` і поклади в неї обидва файли.

`JITO-QA`

`jito-html2json-release-candidate-final.zip`

`JITO_WINDOWS_QA.ps1`

Якщо Windows приховує розширення файлів, у Провіднику вибери Вигляд → Показати → Розширення імен файлів.

Важливо: ZIP має називатися саме `jito-html2json-release-candidate-final.zip`, без `(1)` або `(2)` у назві. Якщо Windows перейменувала його під час завантаження, виправ назву.

Крок 2. Перевір Node.js

Відкрий папку `JITO-QA`, клацни адресний рядок Провідника, введи `powershell` і натисни Enter.

Відкриється вікно PowerShell у потрібній папці.

Введи по черзі:

```
node --version

npm.cmd --version
```

Очікуються номери версій, наприклад `v22.x.x` і `10.x.x`. Конкретні номери можуть відрізнятися.

Якщо з'явиться «node не является внутренней или внешней командой», установи Node.js LTS з офіційного сайта Node.js. Під час установлення залиш увімкненим додавання до PATH, потім закрий і відкрий PowerShell заново.

Для нашого проєкту потрібна версія Node.js 18 або новіша. Сторонніх бібліотек установлювати не потрібно.

Етап 2. Максимальний автоматичний тест

Крок 3. Запусти мій QA-скрипт

У тому самому вікні PowerShell, перебуваючи в папці `JITO-QA`, скопіюй і виконай одну команду:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File ".\JITO_WINDOWS_QA.ps1"
```

Команда запускає лише цей скрипт в окремому процесі PowerShell. Вона не змінює постійних політик системи, не потребує прав адміністратора й не встановлює жодних пакетів. Сам механізм політики запуску описаний Microsoft. [В оригіналі — посилання «Microsoft Learn».]

Скрипт автоматично виконує таку роботу:

1. Перевіряє точний SHA-256 проєкту: `e814416cc0bfb15ff663c09b3c2efcc0682db3df8d894eeff0024e0ddb333fcd`.
2. Перевіряє 46 файлів, наявність README, `index.html`, папки `ai_help` та правильність вихідного JavaScript.
3. Створює нову папку й розпаковує туди проєкт, не змінюючи вихідного ZIP.
4. Запускає `npm test` — очікується 201/201 PASS.
5. Запускає `npm run check` і `npm run benchmark`.
6. Проводить 43 додаткові перевірки: 12 точкових сценаріїв та всі 31 HTML-файл, включно з Unicode, SVG, таблицями, HTML-сутностями, `script`, некоректними закривальними тегами й 10 000 коментарів.

Ці 43 перевірки я попередньо запустив на Node.js у своєму середовищі — 43/43 PASS. Сам PowerShell-скрипт на справжній Windows я звідси запустити не можу: саме його виконання на твоєму комп'ютері нам і потрібно підтвердити.

Крок 4. Подивися результати

Після виконання поряд із файлами з'явиться папка приблизно з такою назвою:

`JITO_WINDOWS_QA_20261028_145000`

У ній будуть:

Файл	Призначення
`QA_SUMMARY.txt`	Головний підсумок усіх перевірок
`01_tests.stdout.txt`	Подробиці 201 тесту
`02_syntax.stdout.txt`	Перевірка синтаксису

`03_benchmark.stdout.txt`	Вимірювання продуктивності
`04_extra_smoke.stdout.txt`	Додаткові перевірки
`BROWSER_CHECKLIST.txt`	Інструкція для Chrome
`project-clean/`	Чисто розпакований проєкт, що перевіряється
`START_HTTP_TEST.cmd`	Запуск локального HTTP-сервера

Наприкінці PowerShell має з'явитися результат:

```
PASS: npm test
Node TAP: tests=201, pass=201, fail=0
PASS: npm run check
PASS: npm run benchmark
PASS: independent smoke + all 31 samples
OVERALL: AUTOMATED PASS - manual Chrome checks still REQUIRED.
```

Якщо з'явиться FAIL, не намагайся нічого виправляти навмання. Надішли мені `QA_SUMMARY.txt` і відповідний файл логу.

Етап 3. Перевірка проєкту як звичайний користувач

Тепер перевіримо те, що могли не виявити автоматичні тести.

Крок 5. Відкрий справжній HTML-файл

Зайди у створену папку результатів, потім у `project-clean`.

Знайди `index.html`, натисни правою кнопкою → Відкрити за допомогою → Google Chrome.

В адресному рядку має з'явитися адреса, що починається з `file:///`.

Ти маєш побачити оригінальний інтерфейс тестового завдання:

```
Jito's Software Developer Intern "html2json" Test Task
Input for "html2json" conversion
Enter your HTML...
Results of "html2json" conversion
Here will be output...
Convert to JSON
Input Example 1
Input Example 2
```

Схематичне розташування елементів оригінального `index.html`, не скриншот.

Важлива особливість оригінального інтерфейсу: кнопки Example 1 та Example 2 заповнюють ліве поле й показують інформаційний JSON. Після вибору прикладу потрібно окремо натиснути Convert to JSON, щоб перевірити результат роботи нашого парсера.

Перевір обидві кнопки прикладів по черзі.

Для Example 1 у результаті мають бути присутні `doctype`, `html`, `head` і `body`. Для Example 2 — `div`, `p`, `button` та `textarea`.

Крок 6. Відкрий Console

У Chrome натисни F12 (на деяких клавіатурах Fn + F12).

Перейди на вкладку Console.

Введи вручну:

```
typeof html2json
```

Очікувана відповідь:

```
'function'
```

Це означає, що Chrome справді завантажив нашу функцію з файла `html2json.js`.

Після натискання Convert не має бути червоних помилок JavaScript на кшталт `Uncaught TypeError`, `ReferenceError` або `SyntaxError`.

Залиш Console відкритою під час подальшого тестування.

Етап 4. Агресивна перевірка вручну

Тепер самостійно вставляй у ліве поле інтерфейсу HTML-приклади нижче й щоразу натискай Convert to JSON.

Я підібрав випадки, які в попередніх версіях спричиняли справжні дефекти.

Примітка укладача: карусель перетворено на послідовний список; усі 10 карток прочитано через штатну кнопку «Следующий». Оригінальні HTML-рядки збережено. Підписи «HTML для вставки в левое поле», «Ожидаемый результат» перекладено як «HTML для вставлення в ліве поле», «Очікуваний результат». Навігаційні кнопки не є технічними повідомленнями.

1. Звичайна вкладеність — сценарій 1 із 10

```
<p>A<b>B</b>C</p>
```

Очікуваний результат: елемент `p` містить три дочірні вузли: текст A, елемент `b` із текстом B, текст C.

2. Unicode та script — сценарій 2 із 10

```
İ<script>x</script><p>after</p>
```

Очікуваний результат: три кореневі вузли: текст İ, елемент `script` та елемент `p`. Нічого не зникає.

3. HTML-сутності — сценарій 3 із 10

```
<p>&notin; &copy &NotEqualTilde;</p>
```

Очікуваний результат: текст перетворюється на символи ∉, © та ≂̸.

4. Таблиці — сценарій 4 із 10

```
<table><tr><td>A<td>B</table>
```

Очікуваний результат: усередині `table` з'являється `tbody`, потім `tr` із двома окремими `td`: A та B.

5. SVG — сценарій 5 із 10

```
<svg><script/><circle cx="1"/></svg>
```

Очікуваний результат: усередині `svg` два елементи: `script` і `circle`. Circle не зникає.

6. Некоректний end tag — сценарій 6 із 10

```
<div>x</div a"><p>after</p>
```

Очікуваний результат: два окремі елементи: `div` і `p`. Текст after зберігається.

7. Складний script — сценарій 7 із 10

```
<script><!--<script>x</script>tail</script><p>after</p>
```

Очікуваний результат: Script містить `<!--<script>x</script>tail`, а наступний `p` містить after.

8. Перевірка відсутності виконання HTML — сценарій 8 із 10

```
<img src=x onerror="alert(1)"><script>alert(2)</script><p>SAFE</p>
```

Очікуваний результат: HTML перетворюється на JSON. Жодного alert-вікна не з'являється; `p` із SAFE присутній.

9. Незакритий textarea — сценарій 9 із 10

```
<textarea><b>Hello</b>&amp;</textarea><p>OK</p>
```

Очікуваний результат: усередині `textarea` текст `<b>Hello</b>&`, після `textarea` окремий `p` з OK.

Примітка укладача: історичну назву «Незакрытый textarea» збережено в перекладі, хоча наведений у повідомленні приклад містить закривальний тег. Назву не виправлено заднім числом.

10. Порожній ввід — сценарій 10 із 10

`(залиш поле порожнім)`

Очікуваний результат: отримуємо об'єкт `{"type":"document","children":[]}`.

Після цих перевірок онови сторінку сполученням Ctrl + R, знову натисни обидві кнопки прикладів і переконайся, що інтерфейс продовжує працювати.

Не вставляй агресивних HTML-прикладів безпосередньо в Console для виконання. Вони призначені виключно для лівого текстового поля застосунку.

Етап 5. Перевірка через HTTP

Ми вже перевірили відкриття локального файла. Тепер перевіримо інший спосіб запуску.

Крок 7. Запусти тестовий сервер

Відкрий папку результатів `JITO_WINDOWS_QA_...` і двічі натисни:

`START_HTTP_TEST.cmd`

З'явиться вікно термінала.

У ньому має бути:

```
Open http://127.0.0.1:8123/index.html
Ctrl+C to stop.
```

Відкрий Chrome і введи адресу:

`http://127.0.0.1:8123/index.html`

Це локальний сервер на твоєму ПК, а не публічний сайт. Скрипт не відкриває доступу з інтернету.

Повтори Example 1, Example 2, Convert і кілька нестандартних HTML-входів. На вкладці Network DevTools перевір, що `index.html` та `html2json.js` завантажуються зі статусом 200.

Після закінчення натисни у вікні сервера Ctrl + C.

Етап 6. Як оцінити результати

Фінальний Windows QA чекліст — початковий стан 0/12:

- [ ] Перевірка SHA-256.
- [ ] 201/201 автоматичних тестів.
- [ ] `npm run check` — PASS.

- [ ] `npm run benchmark` — PASS.
- [ ] 43/43 додаткових smoke-тестів.
- [ ] 31 HTML-файл перевірено.
- [ ] Chrome `file://` — обидві кнопки й Convert.
- [ ] Chrome — 10 ручних HTML-сценаріїв.
- [ ] Console — без необроблених помилок.
- [ ] HTML із `script/onerror` не виконується.
- [ ] HTTP — index і JS завантажуються.
- [ ] Перезавантаження сторінки та повторний Convert.

Що вважати успішним результатом: автоматичні тести пройшли, обидва способи запуску працюють, JSON відповідає очікуваній структурі, червоних JavaScript-винятків немає, HTML-код не виконується в інтерфейсі, а після пошкодженого вводу конвертер продовжує нормально працювати.

Це не дає математичної гарантії відсутності всіх можливих багів. Натомість підтверджує, що саме той архів, який ми збираємося здавати, справді працює на твоїй Windows і витримує найважливіші практичні перевірки.

Що надішли мені після тестування

Прикріпи сюди `QA_SUMMARY.txt`, `01_tests.stdout.txt`, `04_extra_smoke.stdout.txt` і 2–3 скриншоти Chrome — інтерфейс після Convert та вкладку Console. Якщо якийсь етап не пройшов, приклади відповідний `.stderr.txt` і знімок помилки.

Моя мета — щоб перед здачею в нас були не лише звіти ChatGPT Work, а й незалежні справжні докази запуску на твоєму комп'ютері. Далі зможемо оцінити результати й зрозуміти, чи залишилася хоч одна практична проблема, яку необхідно усунути до надсилання Ксенії.

R352 · AI

Аміго, за твоїми скриншотами Windows-перевірка проходить успішно майже за всіма пунктами. Причому тепер це підтверджено на твоєму власному ПК, а не лише в середовищі ChatGPT Work.

Перевірка	Результат
Windows 11 / Node.js 23.11.0	Працює
SHA-256 проєкту та JS	PASS
`npm test`	Exit code 0
`npm run check`	PASS
`npm run benchmark`	PASS
43 додаткові тести	PASS
Інтерфейс Chrome	Працює

Чому ж написано OVERALL: FAIL?

Я відкрив вихідний код нашого `JITO_WINDOWS_QA.ps1` і знайшов причину.

Сам `npm test` завершився з кодом 0, що означає успішне виконання. Але після цього мій QA-скрипт спробував знайти в його виводі рядки формату TAP:

```
# tests 201
# pass 201
# fail 0
```

У твоєму Node.js 23 вивід може використовувати інший формат репортера, а скрипт не передбачив цього випадку. Тому з'явився рядок:

```
WARNING: could not parse TAP summary
```

А потім помилково встановлено загальний статус FAIL.

Це підтверджений недолік мого допоміжного QA-скрипта, а не свідчення помилки в `html2json.js`. Точну кількість успішних тестів на твоєму ПК потрібно додатково прочитати зі звіту.

За рештою скриншотів я бачу коректну вкладеність p/b, окремі елементи після script, правильні tbody/tr/td, збереження SVG circle і декодування HTML-сутностей. У Console функція html2json доступна, а видимих червоних помилок немає.

Як остаточно закрити цей FAIL

Я підготував виправлений QA-скрипт. Він враховує відмінності форматів виводу Node.js і, якщо потрібно, повторно запускає тести з явним TAP-репортером.

Завантажити `JITO_WINDOWS_QA_FIXED.ps1`

Виправлено лише допоміжний Windows QA; вихідний проєкт не змінювався.

Збережи файл поряд із ZIP у `[Вилучено особистий фрагмент, не пов'язаний із тестовим завданням]` і запусти у своєму PowerShell:

Примітка укладача: вилучено лише абсолютний шлях до папки на комп'ютері користувача. Потрібна робоча папка `JITO-QA` й вимога тримати скрипт поряд із ZIP збережені.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File ".\JITO_WINDOWS_QA_FIXED.ps1"
```

Скрипт створить нову папку звітів. Нам потрібен підсумок:

```
Node TAP: tests=201, pass=201, fail=0
OVERALL: AUTOMATED PASS - manual Chrome checks still REQUIRED.
```

Я перевірив синтаксис внесеного виправлення та доступність явного TAP-репортера у своєму середовищі, але запуск виправленого PowerShell на Windows ще належить підтвердити на твоєму комп'ютері.

Що вже підтверджено ручними скриншотами

- Два вихідні приклади відображаються й конвертуються.
- Вкладені HTML-елементи зберігають правильну структуру.
- Unicode не заважає розпізнаванню наступних елементів.
- HTML-сутності декодуються.
- Таблиці отримують tbody, а сусідні td не зникають.
- SVG script не поглинає наступний circle.
- Некоректні закривальні теги та складні стани script відновлюються в показаних випадках.

Зовнішній вигляд сторінки також нормальний — це оригінальний мінімалістичний інтерфейс роботодавця. Нічого переробляти в дизайні не потрібно. Горизонтальне прокручування довгого тексту всередині полів очікуване.

Останні перевірки, які ще потрібні

За надісланими знімками я не можу підтвердити три сценарії: порожній ввід, HTML із onerror/alert, а також відкриття через HTTP-сервер.

Для перевірки безпеки встав у ліве поле:

```
<img src=x onerror="alert(1)">
<script>alert(2)</script>
<p>SAFE</p>
```

Натисни Convert. Очікується JSON, у якому присутні img, script та p, але жодних спливних alert.

Після цього перевір порожній ввід і запусти `START_HTTP_TEST.cmd` із нової папки результатів. Відкрий `http://127.0.0.1:8123/index.html` і повтори обидві кнопки прикладів.

Проміжний вердикт: Windows-тестування виглядає дуже переконливо. Справжніх помилок парсера на надісланих скриншотах я не виявив. Єдина видима автоматична невдача пов'язана з недоліком мого QA-скрипта.

Надішли наступний скриншот PowerShell після запуску виправленого скрипта і, якщо знову виникне помилка, файл `01_tests.stdout.txt`. Тоді остаточно зафіксуємо результат Windows QA, не змінюючи перевіреного коду проєкту.

R354 · AI

Аміго, чудово! Тепер автоматичну перевірку JITO.DEV на твоїй Windows повністю пройдено. Попередній OVERALL: FAIL справді був пов'язаний із помилкою допоміжного QA-скрипта, а не з реалізацією HTML-парсера.

Я уважно вивчив новий скриншот PowerShell.

Підсумок Windows QA — AUTOMATED PASS

Перевірка	Результат
Автоматичні тести	201/201 PASS
Перевірка синтаксису	PASS
Benchmark	PASS
Додаткові тести + 31 приклад	PASS
Цілісність ZIP і JavaScript	PASS
Windows 11 / Node.js 23.11.0	PASS

Найважливіший рядок:

```
Node TAP: tests=201, pass=201, fail=0
```

Це вже справді підтверджений результат на твоєму комп'ютері, а не твердження Work.

Що це означає для jito.dev

Тепер у нас є суттєві технічні підтвердження:

1. Проєкт розпаковується з фінального архіву без пошкоджень.
2. Усі 201 тест проходять на Windows.
3. Перевірено 31 HTML-приклад і додаткові нестандартні випадки.
4. Синтаксичних помилок немає, benchmark проходить.
5. На попередніх скриншотах Chrome ми побачили коректну роботу вихідних прикладів, Unicode, HTML-сутностей, таблиць, SVG і складних станів script.

Жодного підтвердженого нового дефекту самого парсера на Windows поки не виявлено.

Що залишилося перевірити вручну

Сам скрипт правильно пише `manual Chrome checks still REQUIRED`. За твоїми попередніми скриншотами значну частину ручного тестування вже виконано.

Залишилися три короткі перевірки:

- [ ] Порожнє ліве поле → Convert → результат document із порожнім children.
- [ ] HTML із `<script>` і onerror → JSON формується, спливних alert немає.
- [ ] `START_HTTP_TEST.cmd` → `http://127.0.0.1:8123/index.html` → обидві кнопки й Convert працюють.

Історичний стан чекліста: 0 із 3 завершено.

Після них можна буде вважати локальний Windows QA завершеним у межах нашого погодженого набору перевірок.

Мій вердикт

Технічна збірка пройшла автоматичне приймання на Windows.

Це сильний результат перед senior code review. Він не гарантує проходження невідомих тестів роботодавця, але значно знижує ймовірність очевидної помилки під час запуску.

Код зараз не змінюємо. Зберігаємо саме архів із SHA-256 `e814416c...dd333fcd`.

Після трьох залишкових браузерних перевірок наступним пріоритетом стане оформлення повної AI-історії, перевірка публічних посилань і публікація проєкту. Це необхідно, щоб якісну технічну роботу справді допустили до розгляду.

R356 · AI

Залишилося лише 3 ручні перевірки в Google Chrome. Зробимо їх по черзі. Я напишу, куди натискати, що вставляти й який результат має вийти.

Усі перевірки виконуй у тій папці, яку створив останній QA-скрипт:

[Вилучено особистий фрагмент, не пов'язаний із тестовим завданням]

Вилучено абсолютний шлях користувача; технічно потрібна папка результатів `JITO_WINDOWS_QA_20261028_141825` у робочій папці `JITO-QA`.

Перевірка №1. Порожній HTML — найпростіша

1. Відкрий папку `project-clean`.
2. Двічі натисни `index.html` — відкриється Chrome.
3. Натисни ліве поле Input for "html2json" conversion.
4. Натисни Ctrl + A, потім Backspace, щоб видалити весь HTML.
5. Натисни Convert to JSON.

У правому полі має з'явитися такий результат:

```
{
"type": "document",
"children": []
}
```

PASS, якщо вийшов порожній документ без помилок і зависань.

Перевірка №2. Захист від виконання HTML-коду

Тут ми перевіримо, що конвертер сприймає HTML як текстові дані, а не виконує JavaScript із нього.

У тому самому Chrome виділи весь текст у лівому полі (Ctrl + A).

Встав такий HTML:

```
<img src=x onerror="alert(1)">
<script>alert(2)</script>
<p>SAFE</p>
```

Натисни Convert to JSON.

Подивися на праве поле.

- Очікуються три елементи: img, script та p.
- У img буде атрибут onerror із рядковим значенням alert(1).
- У script буде текст alert(2).
- У p буде текст SAFE.

Найголовніше: не має з'явитися жодного спливного вікна із цифрою 1 або 2.

Додатково натисни F12, відкрий вкладку Console й подивися, чи немає червоних помилок JavaScript.

PASS, якщо JSON побудувався, спливних вікон не було й необроблених помилок немає.

Важливо: HTML вставляй лише в ліве поле конвертера, не в Console і не в адресний рядок.

Перевірка №3. Робота через локальний HTTP-сервер

Ми вже переконалися, що застосунок працює за прямого відкриття `index.html`. Тепер перевіримо запуск через HTTP.

Крок 1. Запускаємо сервер

1. Відкрий Провідник Windows (Win + E).
2. Перейди до папки:

[Вилучено особистий фрагмент, не пов'язаний із тестовим завданням]

Повторно вилучено той самий абсолютний шлях; мається на увазі папка результатів останнього QA-скрипта.

3. Знайди файл `START_HTTP_TEST.cmd`.
4. Двічі натисни його.

Має відкритися окреме вікно термінала з повідомленням про запуск локального сервера.

Не закривай вікно сервера, доки тестуєш сайт.

Крок 2. Відкриваємо сайт

Відкрий нову вкладку Chrome (Ctrl + T) і введи:

`http://127.0.0.1:8123/index.html`

Має відкритися знайома сторінка Jito з двома полями й трьома кнопками.

Крок 3. Перевіряємо кнопки

Натисни Input Example 1, потім Convert to JSON. Має з'явитися дерево з html, head, body.

Натисни Input Example 2, потім Convert to JSON. Має з'явитися дерево з div, p, button, textarea.

Встав `<p>Hello Jito!</p>` у ліве поле й натисни Convert to JSON. У правому полі має з'явитися p із текстом Hello Jito!.

Крок 4. Перевіряємо завантаження файлів

1. Натисни F12.
2. Відкрий вкладку Network.
3. Натисни Ctrl + R для перезавантаження сторінки.
4. Знайди рядки `index.html` та `html2json.js`.

У колонці Status обох файлів має бути 200.

PASS, якщо сайт відкрився, кнопки працюють, JSON формується, обидва файли отримали HTTP 200.

Після перевірки можна закрити сервер сполученням Ctrl + C у вікні термінала.

Познач результати

Фінальний Chrome QA — історичний стан 0/3:

- [ ] Порожній HTML → порожній документ.
- [ ] HTML із script та onerror → код не виконується.
- [ ] HTTP → усі кнопки працюють, файли завантажуються.

Надішли мені три скриншоти: результат порожнього вводу, результат перевірки script/onerror та вкладку Network під час відкриття через HTTP. Я перевірю їх і зафіксую остаточний результат Windows QA перед публікацією проєкту.

R358 · AI

Аміго, я уважно перевірив усі 6 нових скриншотів. Результати чудові: останні три браузерні перевірки виконано, і видимих помилок у роботі конвертера я не виявив.

Результати фінального Windows QA

1. Порожній HTML — PASS

На першому скриншоті отримано правильний JSON:

```
{
"type": "document",
"children": []
}
```

2. HTML із script та onerror — JSON PASS

На другому скриншоті правильно збережено img, script та p, включно зі значеннями alert(1), alert(2) і SAFE. Код представлено як дані, а не відображено як виконувану сторінку.

3. Локальний HTTP-сервер — PASS

На скриншотах працює адреса `127.0.0.1:8123`. Обидва приклади й `<p>Hello Jito!</p>` перетворюються коректно. У Chrome DevTools підтверджено відповіді HTTP 200 для `index.html` та `html2json.js`.

Окремо: у Network видно скрипти розширень Chrome — це не файли проєкту й не проблема. Відповідь 204 для `favicon.ico` також не свідчить про помилку парсера.

Загальний підсумок усіх перевірок

Перевірка	Результат
SHA-256 архіву та вихідного JS	PASS
Автоматичні тести	201/201 PASS
Синтаксис	PASS
Benchmark	PASS
Додаткові тести й 31 HTML-приклад	PASS
Перевірка спеціальних HTML-конструкцій	PASS для випробуваних випадків

Відкриття через `file://`	PASS
Відкриття через HTTP	PASS
Обидві кнопки прикладів і Convert	PASS
Порожній ввід	PASS
Небезпечний HTML як дані	Коректний JSON

Одне невелике застереження: скриншот не дозволяє достовірно встановити, що спливне вікно alert жодного разу не з'являлося. Якщо його справді не було, перевірку №2 також повністю пройдено.

Мій вердикт

Локальний Windows QA — успішно пройдено за погодженим набором перевірок.

Тепер ми знаємо, що фінальна збірка з 201 тестом справді працює на твоїй Windows 11, а не лише у звітах AI.

Це сильне підтвердження якості. Водночас воно не гарантує проходження невідомих перевірок сеньйора-розробника й не скасовує відомих обмежень HTML5-парсингу.

Я рекомендую не чіпати вихідного коду. Наступні дії — повна AI-історія, публікація перевіреної збірки на GitHub, перевірка всіх публічних посилань у режимі інкогніто та підготовка до технічного інтерв'ю.

І останнє коротке уточнення: під час перевірки з alert(1) та alert(2) жодних спливних вікон справді не з'являлося? Якщо так, фіксуємо й останню перевірку безпеки як PASS.

R360 · AI

Це дуже просто. Ми перевіряємо, чи виконує браузер JavaScript із введеного HTML, чи просто перетворює його на JSON.

Крок 1. Встав тестовий HTML

Відкрий наш `index.html` у Chrome.

У лівому полі видали старий текст і встав:

```
<img src=x onerror="alert(1)">
<script>alert(2)</script>
<p>SAFE</p>
```

Крок 2. Натисни Convert to JSON

Тепер стеж за екраном.

Правильна поведінка:

- Праворуч з'являється JSON з елементами img, script та p.
- Жодні додаткові вікна не з'являються.
- Сторінка продовжує працювати.

Неправильна поведінка:

Якщо з'являється маленьке спливне вікно з написом 1 або 2 та кнопкою «OK», отже, JavaScript виконався. Це помилка.

Як виглядає небезпечне спливне вікно — схематично:

```
127.0.0.1 повідомляє
1
```

```
OK
```

Це лише приклад зовнішнього вигляду. Таке вікно з'являтися не має.

Крок 3. Перевір ще раз

Натисни Convert to JSON повторно, потім Input Example 1 і знову Convert to JSON.

Якщо жодних спливних вікон не було, JSON формується й кнопки працюють — тест пройдено.

Судячи з твого попереднього скриншота, JSON сформувався правильно.

Тепер відповідай лише на одне запитання: коли ти натискав Convert to JSON із цим HTML, чи з'явилося хоч одне маленьке вікно із цифрою 1 або 2 та кнопкою «OK»?

Якщо ні — ставимо PASS і вважаємо локальну перевірку безпеки завершеною.

R362 · AI

Перевірка безпеки HTML — візуально успішна!

Я уважно подивився твій скриншот.

- Елемент `<img>` коректно перетворено на JSON, атрибут `onerror="alert(1)"` збережено як рядок.
- Вміст `<script>alert(2)</script>` також збережено як текст.
- Елемент `<p>SAFE</p>` присутній.
- Жодних спливних вікон на скриншоті немає.
- Інтерфейс продовжує нормально працювати через HTTP.

Єдине застереження: за нерухомим скриншотом не можна встановити, чи з'являлися вікна `alert` під час натискання Convert. Якщо ти натиснув кнопку й жодних вікон із цифрами `1` або `2` не було, ставимо остаточний PASS.

Підсумок Windows QA

Перевірка	Результат
201 автоматичний тест	PASS
43 додаткові перевірки	PASS
Порожній HTML	PASS
Запуск file:// та HTTP	PASS
Небезпечний HTML	Візуально PASS

Мій висновок: локальне Windows-тестування практично завершено, і підтверджених дефектів у перевірених сценаріях немає. Переписувати код на підставі цих результатів не потрібно.

Наступний справді важливий етап — правильно оформити AI-історію та опублікувати перевірену збірку для jito.dev, не втративши файлів і не порушивши вимог роботодавця.